TCR/BCR Repertoire Downstream Analysis · Corrected & Annotated Edition

Contents

TCR/BCR Repertoire Downstream Analysis — Corrected & Annotated Edition 1
0. About this edition 1
1. Issue list at a glance 2
2. Environment and data 4
3. Part 1 — Obtain clonotype information 6
4. Part 2 — Obtain gene usage information 8
5. Part 3 — Distribution of CDR3 length 12
6. Part 4 — TCR/BCR diversity indices 14
7. Corrected pass and comparison 19
8. Checklist, outputs and summary 26
Appendix A — Deviation log 27
Appendix B — Reproduction 28

TCR/BCR Repertoire Downstream Analysis — Corrected & Annotated Edition

Course module 4 · TBCR analysis (TRUST3 output) Source tutorial: TBCR analysis — Downstream analysis on TRUST output files, Tian Xu & Zexian Zeng, Center for Life Sciences / Center for Quantitative Biology, Academy for Advanced Interdisciplinary Studies, Peking University.

---

0. About this edition

This document is the corrected and annotated edition of the fourth course module. It follows the same protocol as the WES, RNA-seq and RNA-seq-application modules of this repository:

1. the tutorial was executed **as written** on the login node, with no optimisation and no mid-run fixes;
2. every problem encountered or predicted was **recorded, not fixed**, during that run (the issues are listed in § 1 and explained in the Part where they occur);
3. **a separate corrected pass was then run** into `results/05_corrected/` so that the effect of each correction can be measured rather than asserted (the user asked for this explicitly for this module);
4. the documents, scripts, logs and figures are committed together.

Unlike the three previous modules, this one is **not** a sequencing pipeline. It starts from files produced by TRUST3, an assembler that reconstructs TCR/BCR contigs from bulk RNA-seq reads, and it is entirely R-based downstream analysis (`ggplot2`, `dplyr`, `ggpubr`). There is no FASTQ, no alignment and no scheduler involvement: the whole module runs in about one minute on the login node.

What this edition adds on top of the original tutorial:

- the reason each step exists, and what the underlying file actually contains;
- the exact commands and paths used in this run, and the observed numbers;
- nine verified pitfalls (`TBCR-01` to `TBCR-09`), each with the corrected code;
- a full as-written versus corrected comparison over the 375-sample TCGA-OV cohort;
- extension directions that the tutorial only gestures at (TCR beta chain, clinical integration).

---

1. Issue list at a glance

ID	Where	What goes wrong	Correction
TBCR-01	Part 2	`*_report.tsv` begins with a `#count` header line. The tutorial reads it with `read.table()` and no `header = TRUE`, which only works because R's default `comment.char = "#"` silently drops that line. Set `comment.char = ""` anywhere and the header becomes a data row.	Keep the default, or read with `header = TRUE` after stripping the `#` from the first field. Documented, not changed in the as-written run.
TBCR-02	Part 4	`diversity <- nrow(IGH_df)` is the number of **report rows**, not the number of unique CDR3. TRUST splits one CDR3 into several rows when it assigns different V alleles or constant regions: the demo sample has 354 IGH rows but only 345 unique CDR3nt (19 duplicated CDR3nt rows cohort-wide in that one sample). Raw diversity is therefore inflated by roughly 2-3% here.	Collapse by CDR3nt (summing `count`) before counting.
TBCR-03	Part 3	`IGH_df$CDR3aa != "partial"` misses the second non-sequence label, **`out_of_frame`**. The demo sample keeps 2 such rows, whose nucleotide lengths are 46 and 71 — not multiples of three — and they enter the nucleotide length histogram. Across the 375 OV samples there are 9,965 such rows (0.92% of IGH rows).	Filter on a whitelist of the 20 amino acids instead of blacklisting one label.
TBCR-04	Part 3, Part 4	CDR3aa also contains **imputed** sequences written with `?` and `_` placeholders (the `CDR3_score = 0.01` case). The demo sample has 8; the OV cohort has 31,914 (2.94%). They pass the `!= "partial"` filter and are counted as normal amino-acid sequences.	Two filtering tiers: L1 drops `partial` + `out_of_frame`; L2 additionally drops `?`/`_` sequences.
TBCR-05	Part 4	`clonality <- 1 - (entropy / log2(diversity))` divides by zero when a sample has	Return an explicit `NA`, count and report how many samples are undefined.

ID	Where	What goes wrong	Correction
		exactly one clone: `log2(1) = 0` and `entropy = 0`, so the result is `NaN`. In the OV cohort one sample (`TCGA-23-1113`) has raw diversity 1 and gets `NaN`; after correction 9 samples have fewer than 2 valid clones and 3 have none at all. The tutorial writes these `NaN` values straight into the CSV.	
TBCR-06	Part 4	The text defines `H(X) = -sum p_i log p_i` (natural log, nats) but the code uses `log2` (bits). The entropy values are therefore ~1.44 times smaller than the formula implies. Clonality is unaffected because the same base appears in numerator and denominator.	Compute both; report the base used. Corrected pass stores nats and bits.
TBCR-07	Part 2	The `*` placeholder (no D or J call) is treated as a gene. In the demo sample's IGH rows, `*` occurs 60 times in V, 169 in D, 72 in J and 132 in C; in the IGHJ usage table `*` ranks fourth. Usage frequencies are therefore diluted by a non-gene category.	Exclude `*` from the denominator, or report it as "unassigned".
TBCR-08	Part 4	The cohort loop does **not** filter `partial` or `out_of_frame`, while Part 3 does. Across the OV cohort 44.5% of IGH rows are `partial` (40.7%), `out_of_frame` (0.92%) or imputed (2.94%), so Part 3 and Part 4 describe different sequence populations.	Apply one documented filter in both parts (L1 or L2).
TBCR-09	Several	Code hygiene: `aes(x = bcr_metrics_df$diversity)` reaches into the global environment from inside `aes()`; the output file is spelled `diveristy_cor_entropy.png`; two plot blocks draw `p` without `print(p)` (works at top level and under `Rscript`,	Use bare column names in `aes()`, fix the typo in the corrected pass, call `print()`, preallocate, skip the login instructions.

ID	Where	What goes wrong	Correction
		silently produces nothing under `source()`); `bcr_metrics_df <- data.frame()` grows by `rbind` in a loop; Part 0's `ssh/.bashrc` instructions address a different machine.	

Severity ordering: **TBCR-02, 03, 04, 08** change the numbers; **TBCR-05** produces invalid values; **TBCR-06** changes only units; **TBCR-01, 07, 09** are robustness and hygiene issues.

---

2. Environment and data

2.1 Environment

The tutorial's Part 0 sets up conda on a student's own server over SSH and writes a `.bashrc`. That is irrelevant here — the account already exists and the environment is preinstalled — so Part 0 reduces to one line:

```
source /lustre1/share/miniconda3/etc/profile.d/conda.sh
conda activate airr
```

Verified versions in this run:

Component	Version
R	4.3.0 (2023-04-21)
ggplot2	3.4.4
dplyr	1.1.4
ggpubr	0.6.0

Package load emits `package 'x' was built under R version 4.3.3` warnings because the binaries were built under a slightly newer R. They are benign and appear in every log.

2.2 Input data

Everything lives under `/lustre1/share/airr/data/` (read-only course share).

`TRUST_output/` holds the ten output files of **one** sample, `TCGA-02-0055-01A-01R-1849-01`, from TCGA RNA-seq:

File	Size	Content
`*_cdr3.out`	115 KB	one row per consensus assembly: CDR1/2/3, V/D/J/C call, CDR3 score, read support
`*_report.tsv`	88 KB	one row per reported CDR3 with count, frequency, V/D/J/C (VDJTools-compatible)
`*_final.out`	3.3 MB	final contigs with nucleotide weights
`*_raw.out`	2.5 MB	raw contigs with nucleotide weights
`*_assembled_reads.fa`	7.6 MB	reads belonging to each assembly (FASTA)
`*_full.fa`	47 KB	complete TCR/BCR sequences (FASTA)

File	Size	Content
`*_anno.fa`	773 KB	annotation FASTA of the consensus assemblies
`*_cluster.out`	86 KB	cluster membership of consensus assemblies
`*_evo.out`	1.1 KB	evolution information of the final sequences
`*_simplerep.out`	108 KB	simplified report

Only the first two are used by this module: `cdr3.out` for Part 1 (assembly-level) and `report.tsv` for Parts 2-4 (CDR3-level).

`OV_results/` holds 375 `*_report.tsv` files extracted by TRUST3 from the TCGA-OV cohort. Every file has exactly 8 tab-separated fields; verified before running.

2.3 Column layouts

`cdr3.out` has **no header line**. The tutorial assigns the 12 column names by hand:

```
consensus_id, index_within_consensus, V_gene, D_gene, J_gene, C_gene,
CDR1, CDR2, CDR3, CDR3_score, read_fragment_count, CDR3_germline_similarity
```

`report.tsv` **does** have a header line, and it starts with a `#`:

```
#count  frequency  CDR3nt  CDR3aa  V  D  J  C
```

Because `read.table()` defaults to `comment.char = "#"`, that line is skipped and the remaining 930 rows are data (TBCR-01). The tutorial's `colnames()` call then attaches the eight names. Reading the same file with `read.delim(..., comment.char = "")` would produce a bogus first row with `count = "#count"`.

Two sanity checks on the demo report:

- `sum(count) = 6341` reads in total;
- `sum(frequency) = 2`, because the frequency column is normalised **separately for TCR and BCR** (the tutorial says so explicitly) — 1 + 1. This is a useful invariant: if a future TRUST version normalises the whole table, the sum would be 1 and downstream assumptions about per-chain frequencies would change.

`CDR3_score` in the demo sample takes five values, not the three the tutorial describes:

Score	Meaning per tutorial	Demo rows
0	partial CDR3	294
0.5	(not described)	1
0.67	(not described)	30
0.83	(not described)	137
1.00	strongest motif signal	479

2.4 Output layout in this repository

```
results/
├── 01_clonotype/           Part 1  (1 PNG, script, log)
├── 02_gene_usage/          Part 2  (3 PNG, script, log)
├── 03_cdr3_length/         Part 3  (2 PNG, script, log)
├── 04_diversity/           Part 4  (3 PNG + OV_bcr_diversity.csv, script, log)
├── 05_corrected/           corrected pass (9 PNG + 2 CSV + summary, script, log)
└── ref/                    this document and the original tutorial HTML
```

Scripts `setwd()` into their own step directory, so every relative path in the tutorial (`png("...")`, `write.csv(..., "OV_bcr_diversity.csv")`) still writes where the tutorial intends — just inside the repository instead of `~/airr_output`.

---

3. Part 1 — Obtain clonotype information

3.1 What the tutorial does

Load `*_cdr3.out`, attach column names, split the table into TCR (`^TR` prefix in any of V/D/J/C) and BCR (`^IG`), then tabulate how many times each BCR consensus assembly appears and draw the frequency histogram.

3.2 Why this step exists

TRUST3 reconstructs, for each expressed receptor, a **consensus assembly**: several reads that agree on a V-D-J rearrangement are merged into one contig and assigned an identifier (`assemble0`, `assemble1`, …). The `cdr3.out` table is therefore assembly-level: one row per read-cluster member, many rows per assembly. Counting how often an assembly was seen (`Freq` in the tutorial's table) is the first estimate of clonal abundance, and the histogram answers a basic question: is this sample dominated by a few expanded clones, or by a long tail of singletons?

Splitting TCR from BCR is mandatory here because TRUST3 calls both receptor types from the same RNA-seq library, and their gene nomenclatures never mix.

3.3 Command

```
consensus_df <- read.table("/lustre1/share/airr/data/TRUST_output/
TCGA-02-0055-01A-01R-1849-01_cdr3.out", sep='\t')
colnames(consensus_df) <- c("consensus_id","index_within_consensus","V_gene","D_gene",
                            "J_gene","C_gene","CDR1","CDR2","CDR3","CDR3_score",
                            "read_fragment_count","CDR3_germline_similarity")
TCR_df <- consensus_df %>% filter(grepl("^TR", V_gene) | grepl("^TR", D_gene) |
                                  grepl("^TR", J_gene) | grepl("^TR", C_gene))
BCR_df <- consensus_df %>% filter(grepl("^IG", V_gene) | grepl("^IG", D_gene) |
                                  grepl("^IG", J_gene) | grepl("^IG", C_gene))
BCR_consensus_count <- table(factor(BCR_df$consensus_id, levels = unique(BCR_df$consensus_id)))
```

3.4 Observed output

Quantity	Value
`cdr3.out` rows	974
unique `consensus_id`	778
BCR rows / unique BCR assemblies	941 / 745
TCR rows / unique TCR assemblies	33 / 33
rows that are neither TCR nor BCR	0

BCR assembly frequency distribution (895 rows are in assemblies seen 1-5 times, consistent with a long-tailed repertoire):

Times seen	1	2	3	4	5	7	10	11	16	18
Assemblies	659	50	15	7	8	2	1	1	1	1

The `factor(..., levels = unique(...))` construction matters: `table()` on a plain character vector sorts alphabetically and, more importantly, drops nothing here but would drop empty levels; fixing the levels to the observed order keeps the identifier order stable, so `BCR_consensus_df$Var1` lines up with the assembly numbering.

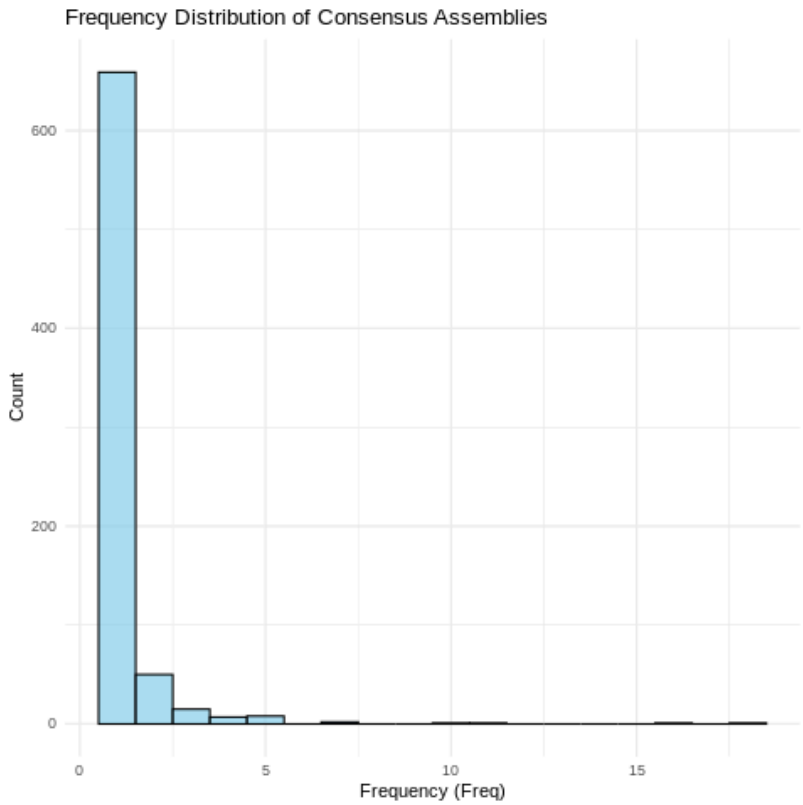


Figure 1 — `BCR_consensus_frequency_distribution.png`: how often each assembled BCR consensus appears in the demo sample.

The shape is the whole point of the panel: 659 of 745 BCR assemblies are seen exactly once, and the tail is a handful of assemblies seen 7-18 times. That long tail is the clonal expansion signal — a repertoire with no expansion would sit almost entirely in the first bar. Two caveats before using it as a clonality measure: `Freq` counts rows (report lines), not reads, so an assembly sequenced deeply and an assembly seen once look identical here; and the x axis is a factor of assembly ids, so the bar order carries no meaning beyond "observed order".

3.5 Pitfalls

- **`table()` returns a factor column.** `as.data.frame(table(...))` gives `Var1` (the assembly id, as a factor) and `Freq`. Any downstream ordering must go through `as.character()` first.
- **`Freq` is a count of rows, not of reads.** The read support of an assembly is `read_fragment_count`, a different quantity; the tutorial never uses it. If you want abundance-weighted clonality from this file, sum `read_fragment_count` instead of counting rows.
- **The TCR table is computed and then ignored.** `TCR_df` is printed and never used again in the module. The demo sample has only 33 TCR assembly rows versus 941 BCR rows, which is typical of a solid tumour: BCR-heavy infiltrate, little TCR clonality without TCR-seq enrichment.

3.6 Tuning

- Filter low-support assemblies (`read_fragment_count >= 2`) before clustering if you want to suppress assembly noise.
- Group near-identical CDR3s (Levenshtein distance 1, or same V-J with 90% CDR3 identity) to collapse clonal families — the tutorial stops at exact-identifier clustering.

3.7 Extensions

- Join `cluster.out` to collapse assemblies into clonal clusters and recompute the histogram at cluster level.
- Join `evo.out` (only 1.1 KB here) to look at somatic hypermutation lineages in the BCR compartment.

---

4. Part 2 — Obtain gene usage information

4.1 What the tutorial does

Read `*_report.tsv`, name the eight columns, extract the TCR alpha chain (`^TRAV`/`^TRAJ`/`^TRAC`) and the BCR heavy chain (`^IGHV`/`^IGHD`/`^IGHJ`/`^IGHC`), then compute IGHV usage, IGHJ usage and the joint V-J distribution, with two bar charts and one heatmap.

4.2 Why this step exists

V and J gene usage is the coarsest description of a repertoire and the one most comparable across samples and studies: it is driven by recombination signal preferences, by selection, and — in tumours — by antigen-driven expansion. V-J pairing adds the combinatorial dimension: a few V-J combinations often carry most of the clonal mass.

The tutorial deliberately works on the **heavy chain** (and, in principle, the beta chain) because those chains determine antigen binding; light chains and the alpha chain are paired partners and are excluded from the usage calculation to avoid double counting the same clone.

4.3 Command

```
air_df <- read.table(".../TCGA-02-0055-01A-01R-1849-01_report.tsv", sep='\t')
colnames(air_df) <- c("count","frequency","CDR3nt","CDR3aa","V","D","J","C")
IGH_df <- air_df %>% filter(grepl("^IGHV", V) | grepl("^IGHD", D) |
                            grepl("^IGHJ", J) | grepl("^IGHC", C))
usage_IGHV_df <- IGH_df %>% group_by(V) %>% summarise(total_count = sum(count)) %>%
  mutate(usage_frequency = total_count / sum(total_count)) %>% arrange(desc(usage_frequency))
```

4.4 Observed output

Quantity	Value
report rows (data)	930
total reads `sum(count)`	6341
`sum(frequency)`	2
TRA rows	6
TRB rows	19
IGH rows	354
IGH reads	2115
distinct IGHV entries (including *)	80
distinct IGHJ entries (including *)	12
distinct V-J pairs	173

Top IGHV by read count:

V gene	reads	usage
IGHV3-15*01	330	0.1560
IGHV5-51*03	243	0.1149
IGHV4-31*03	167	0.0790

Top IGHJ:

J gene	reads	usage
IGHJ5*02	751	0.3551
IGHJ4*02	492	0.2326
IGHJ6*02	478	0.2260

Top V-J pair: `IGHV3-15*01` + `IGHJ5*02`, 288 reads, 0.1362 of IGH mass.

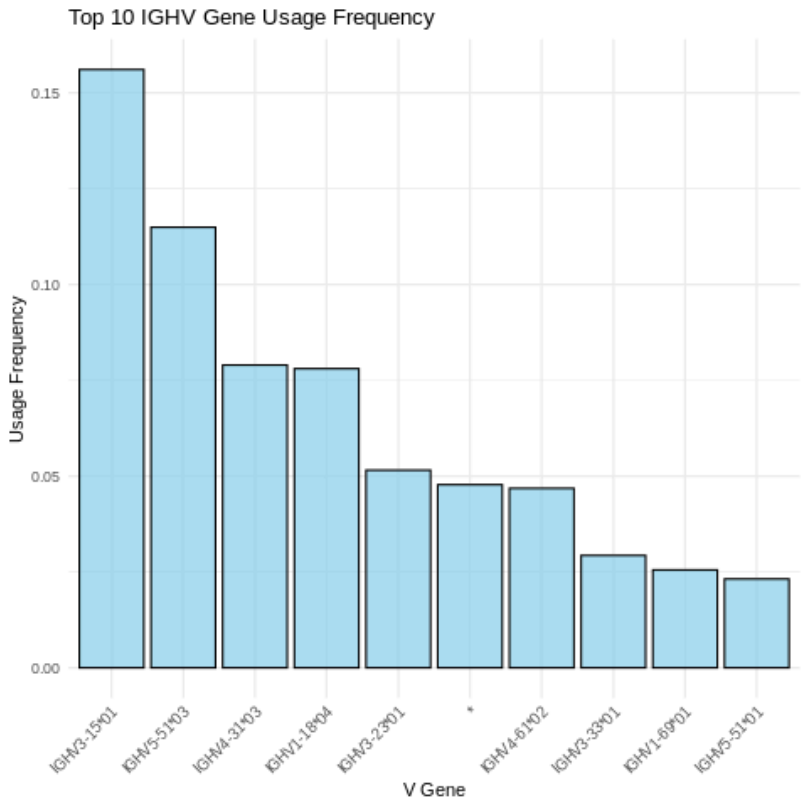


Figure 2 — `top10_IGHV_genes.png`: the ten most-used IGHV alleles by read count.

A bar chart of usage is the fastest way to see whether a sample is skewed: the top allele here (`IGHV3-15*01`) carries 15.6% of IGH reads, and the top ten together dominate the distribution. Read the lengths as evidence about allele not gene usage — `IGHV3-15*01` and a hypothetical `IGHV3-15*04` are separate bars, which makes the table sparse and the top-ten list allele-driven rather than gene-driven.

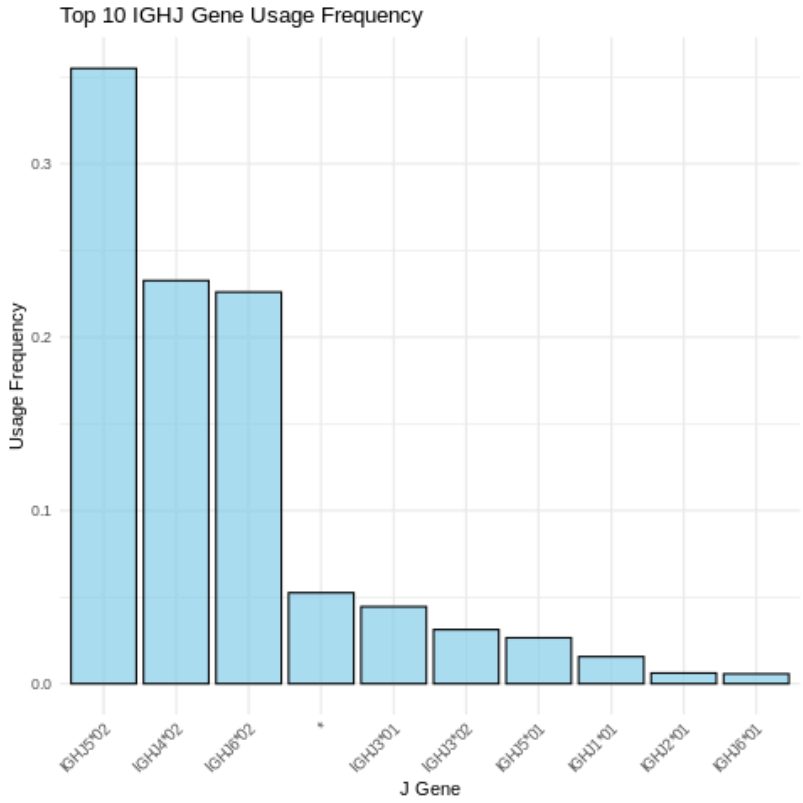


Figure 3 — `top10_IGHJ_genes.png`: the J-segment counterpart.

J usage is far more concentrated than V usage: `IGHJ5*02` alone takes 35.5% of IGH reads, and the top three cover 81%. That asymmetry is expected — there are ~80 V alleles but only ~12 J alleles in use here, so each J carries more mass — and it is the reason a V-J heatmap is informative while a J-only plot is nearly a ranking of six numbers. Note that `*` (unassigned J) would rank fourth: it is absent from this plot only because the tutorial's top-ten cut happens to exclude it.

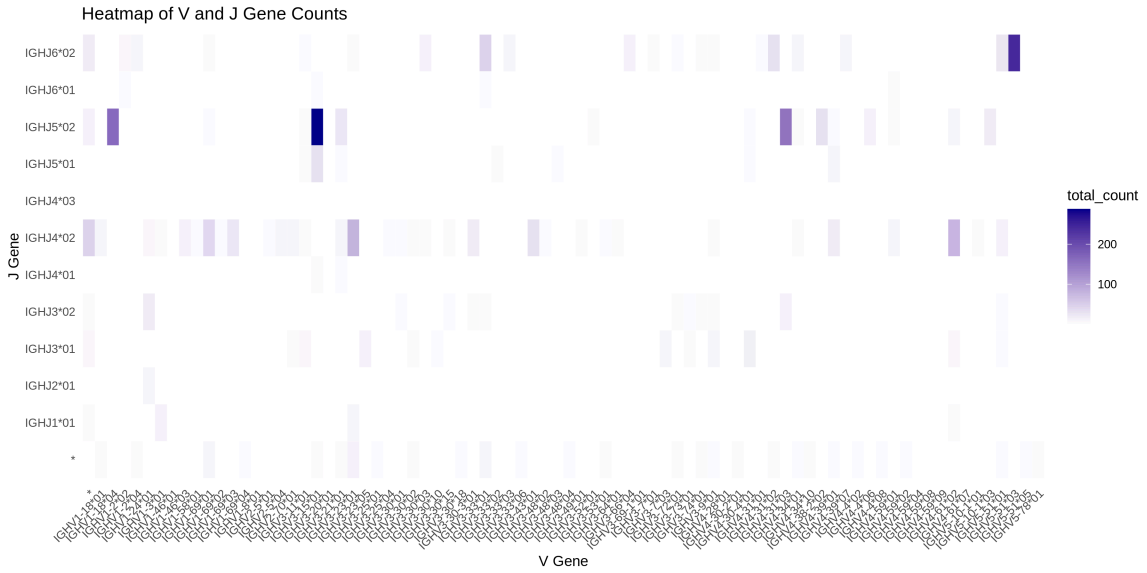


Figure 4 — `heatmap_VJ.png`: V-J pairing counts, J on the rows and V on the columns, colour = `total_count`.

This is the pairing view, and it is also the module's best example of a plot that is technically correct and practically unreadable. The strong cells are real and interpretable — `IGHJ5*02` × a handful of V alleles, `IGHJ6*02` × `IGHV1-69`-type alleles — but eighty allele-level V labels rotated at 45° overlap into a grey band, most of the grid is empty because 173 V-J pairs cannot fill 12 × 80 cells, and the `*` row/column sits at the axes as a reminder that unassigned segments are being treated as genes (TBCR-07). Two one-line fixes make it usable: collapse alleles to gene level (`sub("\\*.*", "", V)`) and drop the `*` category before plotting. Both are left to the corrected pass / extensions rather than silently changed here.

4.5 Pitfalls

- **TBCR-01 (header line).** As discussed in § 2.3, the `#count` line is skipped only because of `comment.char`. This is invisible in the tutorial but becomes a data-corruption bug the moment anyone passes `comment.char = ""` or reads the file with `readr::read_tsv()` (which does not treat `#` as a comment by default and would keep the header as a row).
- **TBCR-07 (`*` is not a gene).** The `*` placeholder marks an unassigned D or J segment. In the demo IGH rows it appears 60 times in V, 169 in D, 72 in J and 132 in C. Because the tutorial groups by the raw column, `*` becomes a category: in the IGHJ usage table it ranks fourth. Every reported usage frequency is therefore slightly diluted, and the V-J heatmap carries a `*` row and column. Excluding `*` changes the IGHJ ranking below the top three but not the top three themselves (verified in the corrected pass).
- **The OR filter is chain-agnostic.** `grepl("^IGHV", V) | grepl("^IGHD", D) | ...` keeps a row if any of the four fields starts with the prefix. That is safe for TRUST output because a row's four fields always belong to one chain, but it is not a guarantee — a mixed row would slip through. A stricter form is to require the V prefix and check consistency.
- **Allele-level granularity.** Genes carry allele suffixes (`IGHV3-15*01`). The tutorial never collapses alleles; two rows differing only by allele are two categories. For cohort comparisons, collapse to gene level (`sub("\\*.*", "", V)`) or the tables become sparse.
- **`summary(total_count)` with `.groups`.** In the V-J block the tutorial passes `.groups = 'drop'`; in the single-gene blocks it does not, so dplyr prints a grouping message. Harmless, but the inconsistency is a hint that these blocks were written at different times.

4.6 Tuning

- Collapse alleles: `mutate(V = sub("\\*.*", "", V))` before grouping.

- Restrict to productive sequences (`CDR3aa` matching the amino-acid whitelist) before computing usage — otherwise `partial` rows contribute V calls without a valid junction.
- Weight by `count` (as the tutorial does) rather than by row: a singleton and a 500-read clone are not equivalent evidence for a V gene.

4.7 Extensions

- Compare usage against a reference repertoire (e.g. the IMGT reference gene distribution) with a chi-square or Jensen-Shannon distance.
- Compute usage separately for expanded versus singleton clones to see whether expansion is V-restricted.

---

5. Part 3 — Distribution of CDR3 length

5.1 What the tutorial does

Drop rows whose `CDR3aa` equals `"partial"`, add `nt_length = nchar(CDR3nt)` and `aa_length = nchar(CDR3aa)`, then draw one histogram per unit.

5.2 Why this step exists

CDR3 length is a quality and biology readout at the same time. Biologically, the junction length distribution is under strong selection (too short loses binding surface, too long is disfavoured by steric constraints) and its mode is species- and chain-specific. Technically, nucleotide length must be a multiple of three for an in-frame junction, so the length distribution is also a quick sanity check on the assembler.

5.3 Command

```
IGH_filter_df <- IGH_df[IGH_df$CDR3aa != "partial",]
IGH_filter_df <- IGH_filter_df %>% mutate(nt_length = nchar(CDR3nt), aa_length = nchar(CDR3aa))
nt_length_df <- IGH_filter_df %>% group_by(nt_length) %>% summarise(total_count = sum(count))
```

5.4 Observed output

Quantity	Value
IGH rows	354
`partial` rows removed	169 (47.7%)
rows kept	185
reads kept	1823 of 2115
nucleotide length range	30-102
nucleotide length mode	66 nt, 477 reads
amino-acid length range	10-34
amino-acid length mode	22 aa, 477 reads
kept rows whose nt length is **not** a multiple of 3	**2**

The 47.7% partial rate is not a defect of the sample: TRUST3 reports a junction whenever it can assemble enough of the CDR3 to call it, and roughly half of the assemblies in this tumour do not cover the full junction. Dropping them is correct for length analysis — but note that it also drops half the reads, which is why Part 4 must make an explicit decision about the same sequences (TBCR-08).

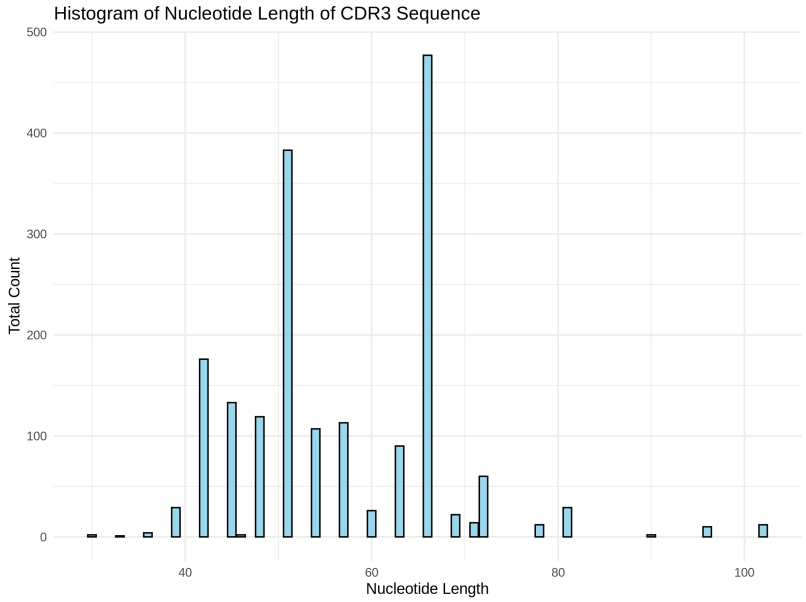


Figure 5 — `CDR3_nt_length_distribution.png`: CDR3 nucleotide length after removing `partial` rows only.

The distribution is close to what a functional repertoire should look like — mode 66 nt (22 codons), range 30-102, and a smooth bell around the mode. The two bars that break it are visible once you look for them: the **46 nt** and **71 nt** rows, which are the two `out_of_frame` assemblies that survived the `!= "partial"` blacklist (TBCR-03). They sit at lengths that are not multiples of three, i.e. physically impossible as in-frame junctions, and they are exactly why the filter should be a whitelist on amino acids rather than a blacklist on one label.

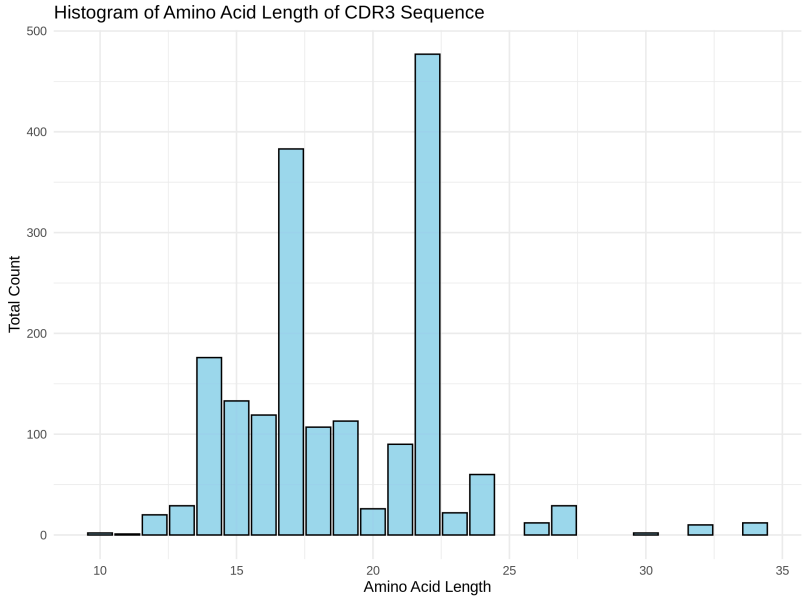


Figure 6 — `CDR3_aa_length_distribution.png`: the same rows in amino acids.

This panel shows the same two artefacts from the other side: `nchar()` counts the `?` and _ placeholders of the eight imputed sequences as residues (TBCR-04), so their lengths are one or two residues too long, and the out-of-frame rows land on non-integer-looking positions relative to the bell. The biological read

is otherwise standard: 10-34 aa, mode 22, i.e. the canonical CDR3 loop length of a human BCR. Keep the figure and the filter decision together — the plot alone cannot tell you whether the tail is biology or bookkeeping.

5.5 Pitfalls

- **TBCR-03 (`out_of_frame` survives the filter).** The two rows with non-multiple-of-three nucleotide lengths are precisely the `out_of_frame` rows: 46 nt and 71 nt. They are not `"partial"`, so the tutorial's blacklist keeps them, and they appear as two isolated bars in the nucleotide histogram. The correct filter is a whitelist: keep only sequences matching `^[ACDEFGHIKLMNPQRSTVWY]+$`.
- **TBCR-04 (imputed sequences).** Eight IGH rows carry `?` or `_` inside the amino-acid string (e.g. `CATTMIRDYFYYGMDV_`, `CAKPGDYDSSSYF?QW`). `nchar()` counts those placeholders as residues, so they enter the length distribution as if they were real amino acids. Whether to keep them is a judgement call — the nucleotide sequence is complete but partly imputed — but the choice must be stated.
- **`nchar()` on a factor.** If `CDR3aa` were read as a factor (it is not here, because `read.table` defaults to `stringsAsFactors = FALSE` since R 4.0), `nchar()` would return the length of the level index. Worth remembering when porting the code to older R.
- **No zero-length guard.** A sample with no surviving row yields an empty histogram and `ggplot` errors rather than an empty plot.

5.6 Tuning

- Plot the same distribution after L2 filtering (whitelist) to see how much the two `out_of_frame` rows and the eight imputed rows actually move it: in the demo sample the mode stays at 66 nt / 22 aa with 474 instead of 477 reads, so the effect is negligible here but not necessarily in other samples.
- Normalise by total reads to compare samples with different sequencing depth.

5.7 Extensions

- Compare the observed length distribution with the theoretical distribution of a random V-D-J recombination (a null model) to detect selection.
- Stratify by V gene family: long CDR3s are enriched in certain V genes.

---

6. Part 4 — TCR/BCR diversity indices

6.1 What the tutorial does

Defines five indices, computes them by hand for the demo sample, then loops over the 375 OV files, writes `OV_bcr_diversity.csv`, and draws three Spearman correlation plots.

6.2 The indices, and what each one measures

Let a repertoire have `N` unique sequences with read counts `c_i`, total `C = sum c_i`, and frequencies `p_i = c_i / C`.

Index	Formula in the tutorial	What it captures
raw diversity	number of unique CDR3	**richness** only; ignores abundance
CPK	`(unique CDR3 / total reads) * 1000`	richness normalised by sequencing depth
entropy	`H = -sum p_i log p_i`	richness and evenness together
clonality	`1 - H / log N`	how far the repertoire is from a uniform distribution of its `N` sequences; 0 = perfectly even, 1 = one clone dominates

Index	Formula in the tutorial	What it captures
Gini-Simpson	`1 - sum p_i^2`	probability that two randomly drawn reads come from different clones

The tutorial's note that only the heavy (BCR) and beta (TCR) chains are analysed is a scope decision, not a mathematical one: those chains determine binding, and including both chains of a pair would count each clone twice.

6.3 Command

```
diversity <- nrow(IGH_df)
CPK <- diversity/sum(IGH_df$count) *1000
freqs <- IGH_df$count/sum(IGH_df$count)
entropy <- -sum(freqs * log2(freqs))
clonality <- 1 - (entropy/log2(diversity))
gini <- 1 - sum(freqs * freqs)
```

followed by the 375-file loop:

```
for (file in files) {
  air_df <- read.table(paste0("/lustre1/share/airr/data/OV_results/",file), sep="\t")
  colnames(air_df) <- c("count","frequency","CDR3nt","CDR3aa","V","D","J","C")
  IGH_df <- air_df %>% filter(grepl("^IGHV", V) | grepl("^IGHD", D) |
                              grepl("^IGHJ", J) | grepl("^IGHC", C))
  diversity <- nrow(IGH_df); CPK <- diversity/sum(IGH_df$count) *1000
  freqs <- IGH_df$count/sum(IGH_df$count)
  entropy <- -sum(freqs * log2(freqs))
  clonality <- 1 - (entropy/log2(diversity))
  gini <- 1 - sum(freqs * freqs)
  bcr_metrics_row <- data.frame(diversity, CPK, entropy, clonality, gini)
  rownames(bcr_metrics_row) <- tools::file_path_sans_ext(basename(file))
  bcr_metrics_df <- rbind(bcr_metrics_df, bcr_metrics_row)
}
write.csv(bcr_metrics_df, "OV_bcr_diversity.csv")
```

6.4 Observed output

Demo sample, as written:

Index	Value
diversity	354
CPK	167.3759
entropy (log2)	6.3314
clonality	0.252280
Gini-Simpson	0.956772

Cohort (375 samples, 13.9 s):

Index	Range	Median
diversity	1 - 26,978	918
CPK	15.567 - 1000	180.7188
entropy (log2)	0 - 13.2401	—
clonality	0.0195 - 0.7306	—
Gini-Simpson	0 - 0.9995	—
`NaN` values	1 (clonality, `TCGA-23-1113`)	—

Spearman correlations, as written:

Pair	rho
diversity vs entropy	0.7957
CPK vs entropy	−0.2470
CPK vs clonality	−0.8790

The sign pattern is the expected one: richness and entropy move together; CPK is high in samples that are deep but not necessarily diverse (it is a ratio to reads, so a small sample with few reads can reach CPK 1000); CPK and clonality are strongly anti-correlated because a repertoire dominated by one clone has few unique CDR3s per read.

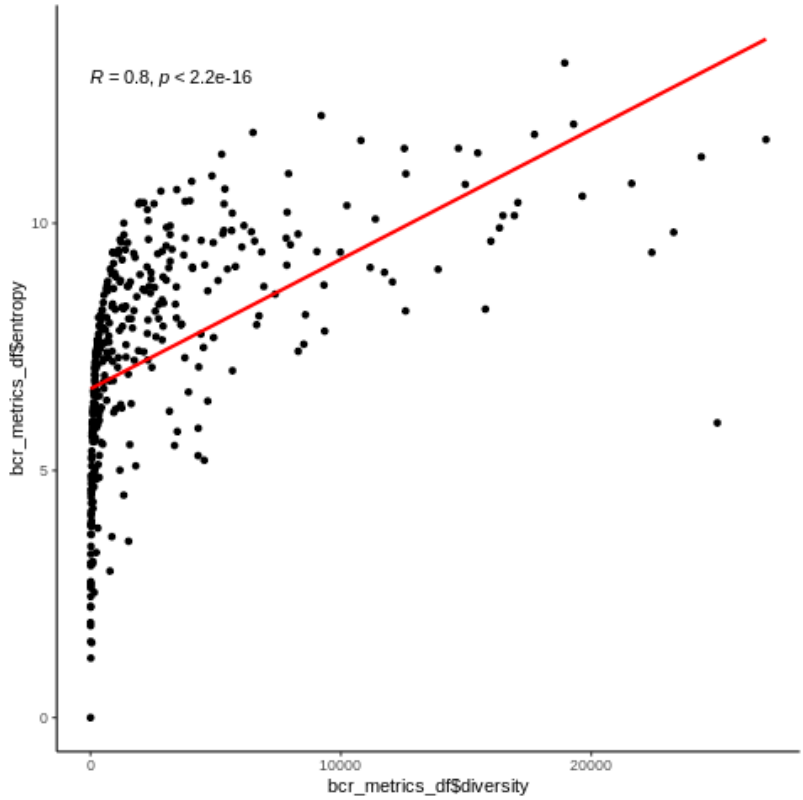


Figure 7 — `diveristy_cor_entropy.png` (the filename typo is the tutorial's): diversity against Shannon entropy across the 375 TCGA-OV samples.

The tight monotone relationship (Spearman 0.7957 as written) is a consistency check rather than a discovery: both quantities measure richness, one as a count and one as a weighted sum, so a sample cannot have many unique CDR3s and low entropy. The scatter around the line is the interesting part — samples that fall below it have many singletons (high richness, low evenness), which is the signature of a repertoire assembled from low-support sequences.

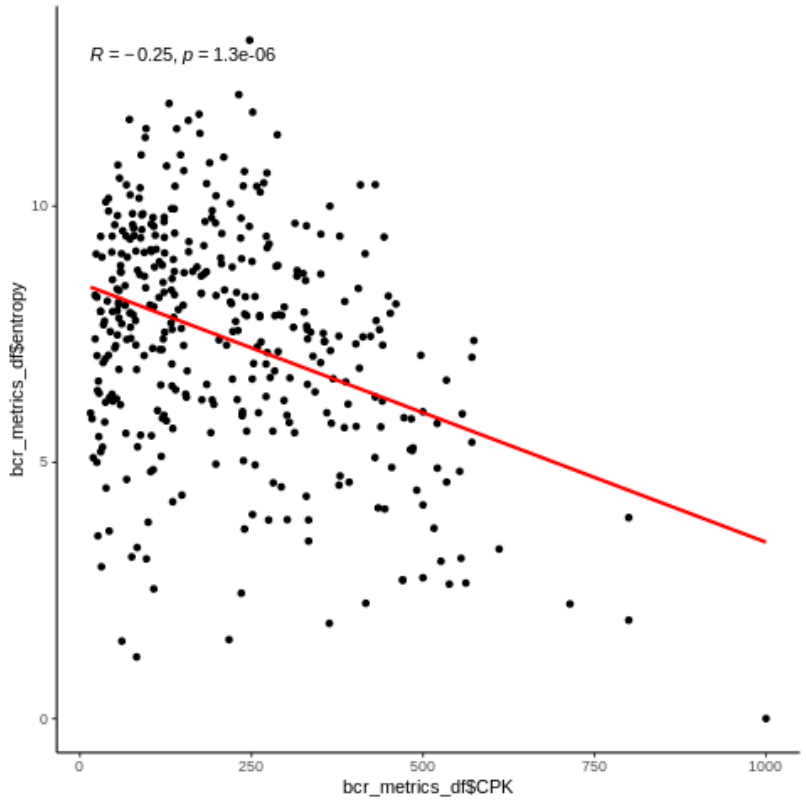


Figure 8 — `CPK_cor_entropy.png`: clones per thousand reads against entropy.

Much weaker (rho −0.2470) and rightly so: CPK normalises by reads while entropy normalises by clone frequencies, so a deeply sequenced but evenly distributed sample and a shallow but clonal sample can land on the same CPK. A weak correlation here is not a failed analysis; it is the evidence that CPK and entropy are not measuring the same thing despite both being “diversity” numbers.

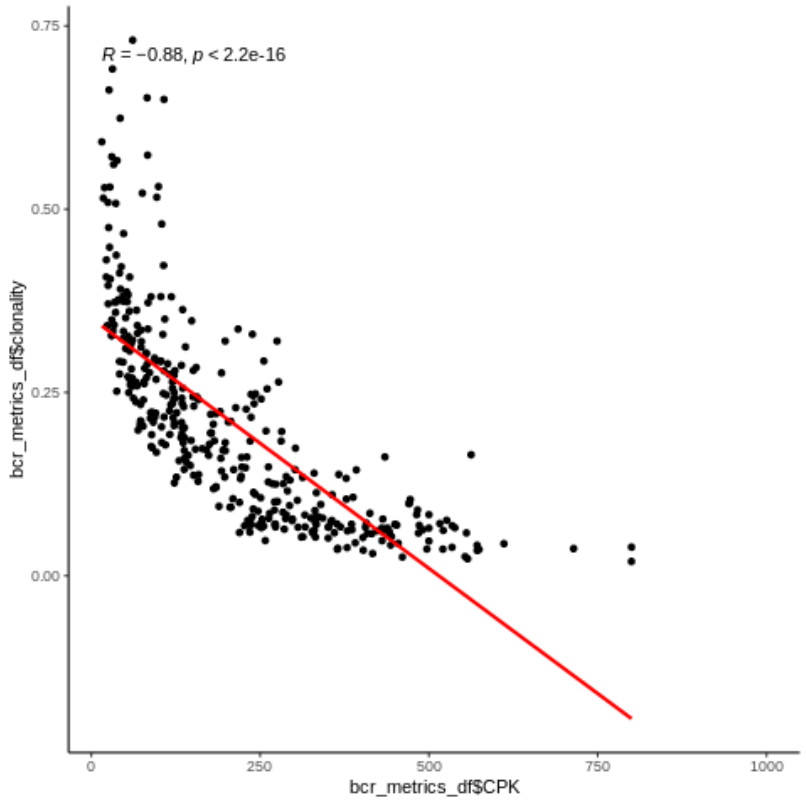


Figure 9 — `CPK_cor_clonality.png`: CPK against clonality, the strongest of the three relationships.

The funnel is the message: at low CPK the clonality values spread from 0 to 0.7, and as CPK grows the cloud collapses onto a floor near zero, giving rho −0.8790. Notice also what the red fit does at the right edge — it extrapolates **below zero**, which is impossible for `1 − H/log(n)`; a linear model on a bounded index is fine for describing a trend and wrong for predicting an individual sample. One point (`TCGA-23-1113`) is missing from this panel because its single clone makes clonality `NaN` in the as-written code (TBCR-05).

6.5 Pitfalls

- **TBCR-02 (diversity is `nrow`, not unique CDR3).** TRUST's report is keyed by the full V/D/J/C assignment, so one CDR3 can appear on several rows. The demo sample: 354 rows, 345 unique CDR3nt, and only 173 unique CDR3aa (the amino-acid level collapses synonymous variants). `nrow()` therefore overstates richness by 2.6% here; cohort-wide the median overstatement is about 2x after also correcting the sequence-quality filter (§ 7).
- **TBCR-05 (`NaN` for single-clone samples).** With `N = 1`, `entropy = 0` and `log2(N) = 0`, so `clonality = 1 - 0/0 = NaN`. One OV sample hits this in the as-written run. Writing `NaN` into the CSV is worse than writing `NA`: many tools propagate `NaN` silently through means and correlations.
- **TBCR-06 (log base).** The formula uses `log` (nats), the code uses `log2` (bits). Entropy values differ by a factor `ln 2 = 0.6931`. Clonality is unaffected, because both numerator and denominator use the same base — this is exactly why the normalised form is the portable one.
- **TBCR-08 (no sequence filter in the loop).** Part 3 removed `partial`, the loop does not, so the entropy of a sample is computed over a mixture of full-length and partial junctions. Across the OV cohort, 40.7% of IGH rows are `partial`, 0.92% `out_of_frame` and 2.94% imputed. This is the single largest driver of the discrepancy between the as-written and corrected diversity values.

- **Frequency vs count.** The loop recomputes frequencies from `count` and ignores the `frequency` column. That is correct (the given column is per-chain normalised), but it means the two columns are never cross-checked.
- **`rbind` in a loop.** 375 iterations is fine; 10,000 would not be. Preallocate or use `dplyr::bind_rows` on a list.
- **TBCR-09 (`aes()` with `$`).** `aes(x = bcr_metrics_df$diversity)` evaluates the global vector rather than the data frame's column. It works until the data frame is filtered or renamed; `aes(x = diversity)` is both shorter and safer.

6.6 Tuning

- If `N` is small, clonality is unstable (it divides by `log N`). Report it only for samples above a minimum clone count (the corrected pass flags the 9 samples with fewer than 2 valid clones).
- Use `vegan::diversity` or `immunarch` if you want vetted implementations and confidence intervals.
- Downsample reads to a common depth before comparing CPK across samples; CPK is depth-normalised but not depth-invariant when richness saturates.

6.7 Extensions

- **TCR beta chain.** The tutorial's own rationale (analyse the chain that determines binding) applies equally to TRB. The corrected pass computes TRB diversity with the same rules: 369 of 375 samples have at least one valid TRB clone, median diversity 38, range 0-529 (see `OV_trb_diversity_corrected.csv`).
- **Clinical integration.** The tutorial's last sentence suggests joining clinical annotations and comparing indices between groups. TCGA-OV clinical data are available from the GDC; the comparison is a Wilcoxon test or a Cox model with diversity as a continuous covariate.
- **Rarefaction curves.** Plot diversity against sequencing depth per sample to separate biological richness from technical sampling.

---

7. Corrected pass and comparison

The corrected pass lives in `results/05_corrected/` (`scripts/corrected_analysis.R`). It recomputes the as-written indices with the identical code path, then applies the corrections, so that every difference below is attributable to a named fix rather than to a different implementation.

7.1 Definitions used

Tier	Sequence filter	Clone definition
raw	none (as written)	one report row
L1	drop `partial` and `out_of_frame`	one unique CDR3nt, counts summed across rows
L2	additionally drop `?`/`_` imputed sequences	one unique CDR3nt, counts summed

Entropy is reported in both nats (matching the formula) and bits (matching the code); clonality is base-invariant.

7.2 Demo sample

Index	raw	L1	L2
diversity	354	177	169
CPK	167.3759	97.9524	94.0980
entropy (nats)	4.3886	3.7572	3.7308
entropy (bits)	6.3314	5.4205	5.3824

Index	raw	L1	L2
clonality	0.2523	0.2741	0.2727
Gini-Simpson	0.9568	0.9393	0.9385

The demo sample has 354 IGH rows, 169 `partial`, 2 `out_of_frame`, 8 imputed and 175 valid; L1 keeps 183 rows which collapse to 177 unique CDR3nt.

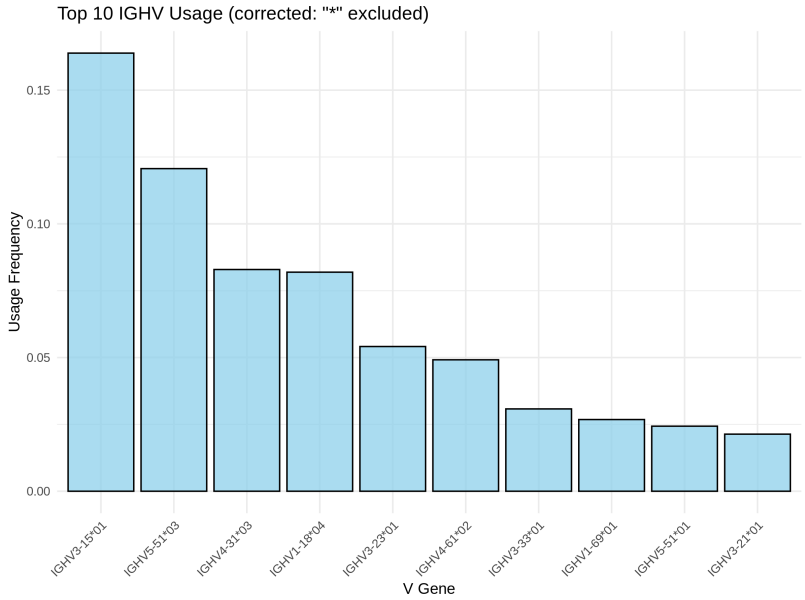


Figure 10 — `corrected_top10_IGHV_genes.png`: IGHV usage with the `*` placeholder excluded.

Compare with Figure 2: the top five are identical (`IGHV3-15*01`, `IGHV5-51*03`, `IGHV4-31*03`, `IGHV1-18*04`, `IGHV3-23*01`), the table simply has one fewer entry (80 → 79) and every percentage is computed against a denominator that no longer contains reads with no V call. The point of showing both is that TBCR-07 changes the normalisation, not the biology — which is why it is a cheap correction that should still be made explicit.

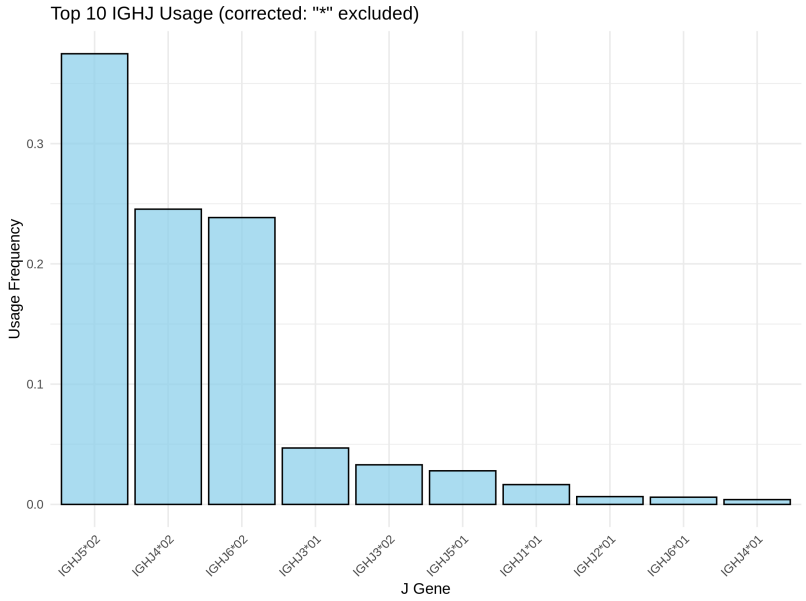


Figure 11 — `corrected_top10_IGHJ_genes.png`: the J counterpart under the same denominator.

Same story on the J side: `IGHJ5*02` (35.5%), `IGHJ4*02` (23.3%) and `IGHJ6*02` (22.6%) are unchanged, the ranking below them shifts slightly because `*` no longer occupies a slot. If a downstream claim depended on "the fourth most used J segment", the correction would change it; if it depended on the top three, it would not.

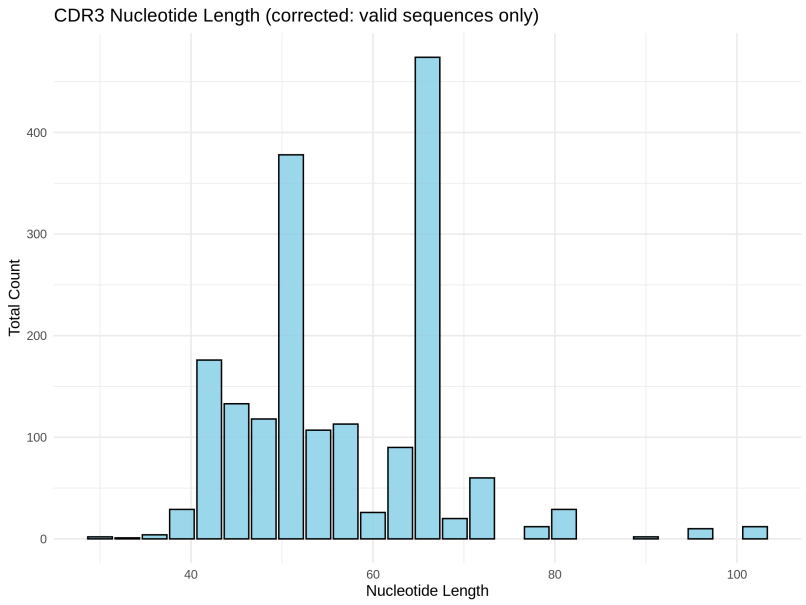


Figure 12 — `corrected_CDR3_nt_length_distribution.png`: nucleotide lengths under the L2 filter (175 rows).

The two impossible bars are gone. Every retained length is now a multiple of three, the mode stays at 66 nt with 474 reads, and the range 30-102 nt is unchanged — so the correction removed exactly the two `out_of_frame` rows and nothing else. This is what "a filter that does what its comment says" looks like: the bell does not move, the impossible spikes disappear.

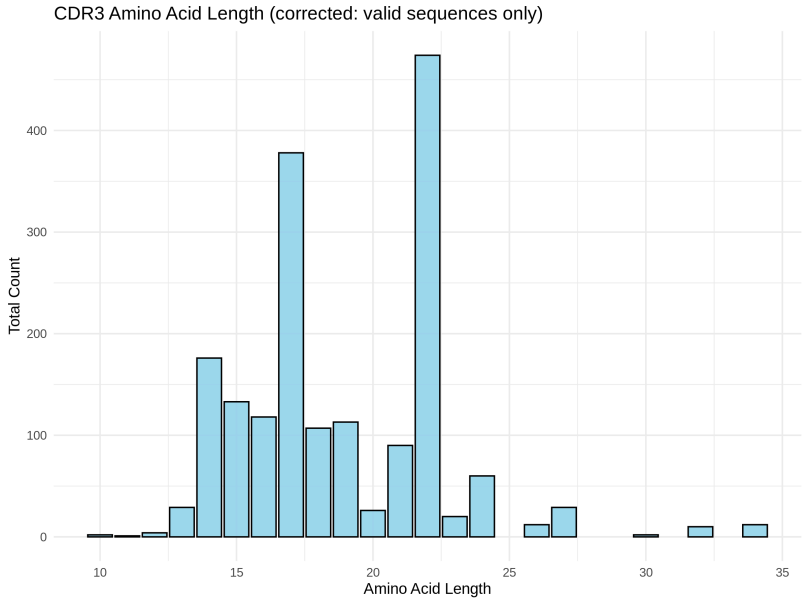


Figure 13 — `corrected_CDR3_aa_length_distribution.png`: the amino-acid view under L2.

With `partial`, `out_of_frame` **and** the eight imputed sequences removed, the distribution is 10-34 aa with mode 22 aa. The visible change against Figure 6 is at the edges: the imputed sequences were inflating individual counts by one or two residues, which matters when the statistic of interest is "the fraction of CDR3s longer than 20 aa" rather than the mode.

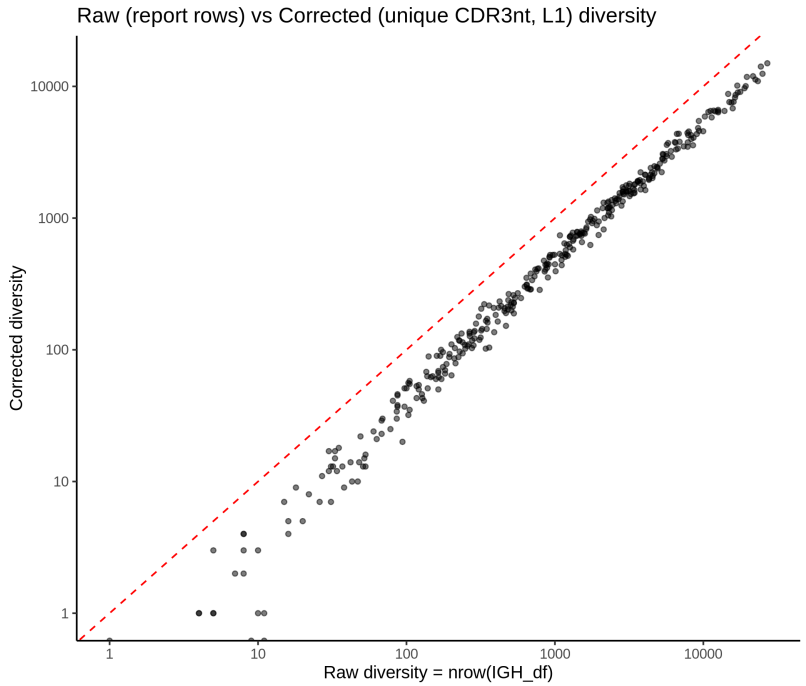


Figure 14 — `raw_vs_corrected_diversity.png`: raw diversity (report rows) against corrected diversity (unique CDR3nt, L1), log-log, with y = x drawn.

Every one of the 375 samples falls **below** the identity line, which is the single most important consequence of the correction: the as-written index is an upward-biased count, never an unbiased one, because TRUST splits one CDR3 into several rows when it assigns different V alleles or constant regions. The bias is proportionally larger at low diversity (about 2× for repertoires with <10 clones of raw diversity) and compresses towards ~1.2× at 10,000+ — i.e. it distorts exactly the samples where clonality claims are made. The points sitting on the bottom edge (corrected = 1, raw = 6-10) are the samples that collapse to a single clone, the ones whose clonality is undefined in § 7.4.

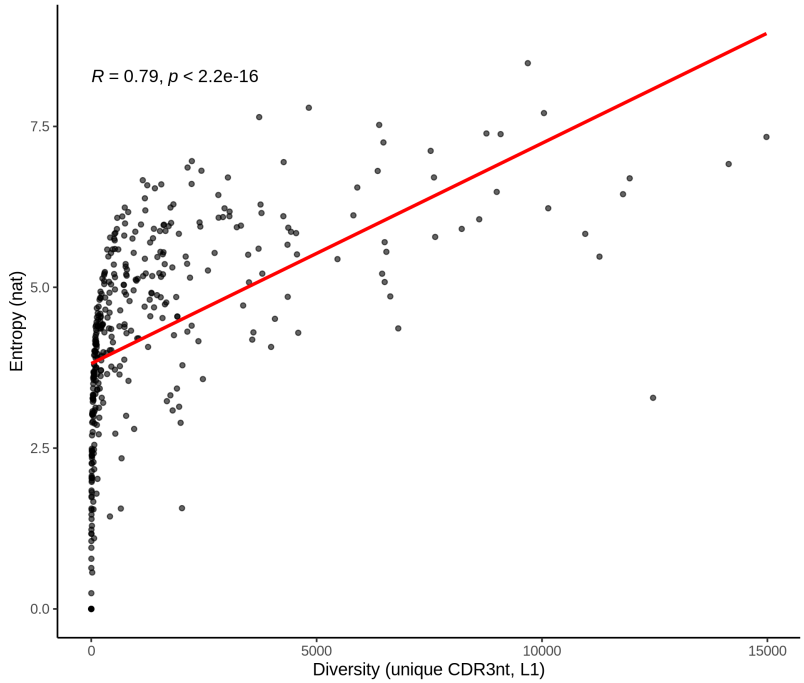


Figure 15 — `corrected_diversity_cor_entropy.png`: the richness-entropy relationship after correction.

Same shape as Figure 7, slightly tighter and with a lower x range, because the count axis halves. The correlation barely moves (0.7957 → 0.7862), which is the reassuring pattern of a scale correction: the ordering of samples is preserved (Spearman 0.9975 between raw and L1 diversity) even though every absolute number changed. Contrast this with a correction that changes ranks — that would be a red flag about the correction, not about the data.

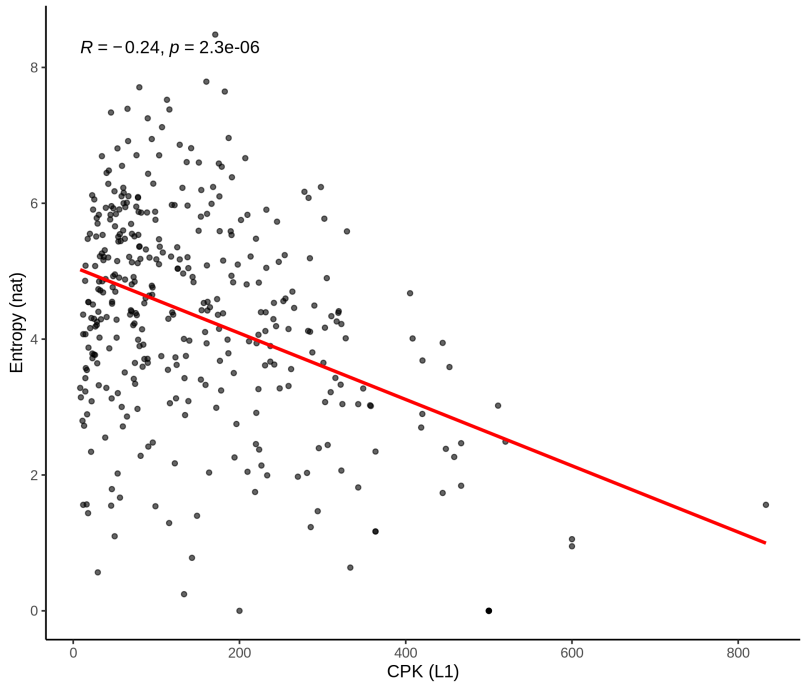


Figure 16 — `corrected_CPK_cor_entropy.png`: CPK against entropy under L1.

The weakest of the three relationships survives at −0.2421 (from −0.2470). Read it as evidence that CPK's normalisation by reads is what decouples it from entropy — after removing partial and duplicated rows, both quantities changed by different factors but stayed just as weakly related, so the weak relation is structural rather than an artefact of the counting bug.

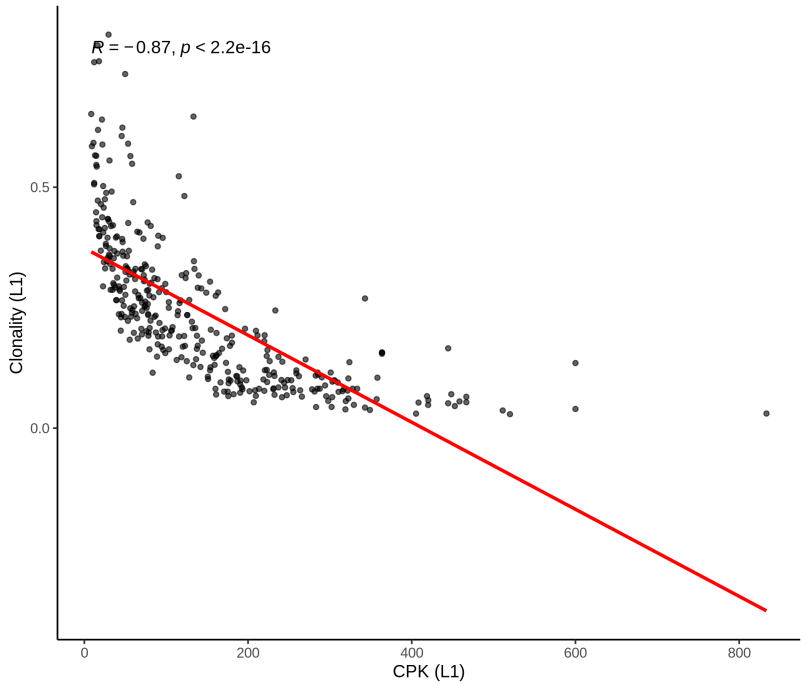


Figure 17 — `corrected_CPK_cor_clonality.png`: CPK against clonality under L1, the strongest relation of the set.

rho −0.8729 (from −0.8790): the correlation is robust, the cloud is not identical. After correction the clonality values sit higher on average (median 0.1843 → 0.2196, i.e. removing duplicated rows makes each sample look more clonal, because the collapsed clone counts concentrate the mass), and the funnel is slightly narrower at low CPK. The three samples with no valid clone are absent rather than plotted at zero — the corrected pass returns `NA` instead of a fabricated value (TBCR-05).

7.3 Cohort (375 samples)

Index	raw median	L1 median	L1 / raw	Spearman(raw, L1)
diversity	918	446	0.484	0.9975
CPK	180.72	113.35	0.640	0.9877
clonality	0.1843	0.2196	1.172	0.9690
Gini-Simpson	0.9820	0.9645	0.985	0.9672

Ranges, raw to L1:

Index	raw	L1
diversity	1 - 26,978	0 - 14,979
CPK	15.567 - 1000	8.408 - 833.333
clonality	0.0195 - 0.7306	0.0290 - 0.8168
Gini-Simpson	0 - 0.9995	0 - 0.9991

Interpretation: the **absolute** values change substantially (diversity roughly halves, because half the rows are partial and the rest are collapsed), but the **ranking** of samples is essentially preserved (Spearman 0.9975). Downstream conclusions of the form “sample A is more diverse than sample B” therefore survive; conclusions of the form “this cohort has a median diversity of 918” do not, because that number is an artefact of counting partial rows. This is the practical argument for the corrected definition.

7.4 Undefined values

Tier	NA in clonality	NA in entropy / CPK / Gini	samples with fewer than 2 clones	samples with no valid clone
raw	1	0	1	0
L1	9	3	9	3
L2	9	3	9	3

The three samples with no valid IGH clone (TCGA-13-0800, TCGA-23-1113, TCGA-24-1419) have all indices undefined. They should be excluded from cohort statistics explicitly, not silently imputed.

7.5 Correlation structure

Pair	raw	L1	L2
diversity vs entropy	0.7957	0.7862	0.7875
CPK vs entropy	−0.2470	−0.2421	−0.2393
CPK vs clonality	−0.8790	−0.8729	−0.8722

The correlation structure is robust to the correction — another sign that the corrected pass changes the scale, not the biology.

7.6 Other corrections in the same pass

- **Gene usage** excludes *: the IGHV table drops from 80 to 79 entries, and the top five V genes are unchanged (IGHV3-15*01, IGHV5-51*03, IGHV4-31*03, IGHV1-18*04, IGHV3-23*01); the top three IGHJ genes are unchanged (IGHJ5*02, IGHJ4*02, IGHJ6*02).
- **CDR3 length** on L2: 175 rows, all nucleotide lengths multiples of three (the two out_of_frame rows are gone), range 30-102 nt, mode 66 nt / 22 aa with 474 reads.
- **Plots** use bare column names in aes(), call print(), and fix the filename typo in the corrected copies only (the as-written typo diveristy_cor_entropy.png is kept for fidelity).

7.7 What the corrected pass does not do

- It does not re-run TRUST3 or question the assembler's calls.
- It does not collapse V alleles in the usage tables (that is left as an extension).
- It does not add clinical covariates, because the tutorial provides none.
- It does not silently replace NaN with a number: undefined stays NA.

---

8. Checklist, outputs and summary

8.1 Checklist

Check	Expected	Observed
ck1	environment airr activates and loads ggplot2/dplyr/ggpubr	yes (R 4.3.0, 3.4.4 / 1.1.4 / 0.6.0)
ck2	demo cdr3.out has 12 columns, no header	yes, 974 rows
ck3	TCR and BCR rows partition the table	yes, 33 + 941 = 974, 0 leftovers
ck4	demo report.tsv has 8 columns	yes, 930 data rows after the # header is skipped
ck5	report frequency sums to 2 (TCR and BCR each normalised)	yes
ck6	IGH usage frequencies sum to 1	yes

Check	Expected	Observed
ck7	CDR3 length histogram shows the expected mode	yes, 66 nt / 22 aa
ck8	375 OV samples processed	yes, 13.9 s
ck9	diversity CSV has one row per sample	yes, 375
ck10	three correlation plots produced	yes
ck11	`NaN` documented rather than hidden	yes, 1 sample, plus 9 undefined after correction
ck12	corrected pass reproduces raw indices exactly	yes, demo raw row matches Part 4 to six decimals
ck13	corrected vs raw ranking preserved	yes, Spearman 0.9975 for diversity

8.2 Output inventory

```
01_clonotype/    BCR_consensus_frequency_distribution.png, log_part1.txt, scripts/part1_clonotype.R
02_gene_usage/   top10_IGHV_genes.png, top10_IGHJ_genes.png, heatmap_VJ.png,
                 log_part2.txt, scripts/part2_gene_usage.R
03_cdr3_length/  CDR3_nt_length_distribution.png, CDR3_aa_length_distribution.png,
                 log_part3.txt, scripts/part3_cdr3_length.R
04_diversity/    OV_bcr_diversity.csv, diveristy_cor_entropy.png, CPK_cor_entropy.png,
                 CPK_cor_clonality.png, log_part4.txt, scripts/part4_diversity.R
05_corrected/    OV_bcr_diversity_corrected.csv, OV_trb_diversity_corrected.csv,
                 corrected_summary.txt, raw_vs_corrected_diversity.png,
                 corrected_diversity_cor_entropy.png, corrected_CPK_cor_entropy.png,
                 corrected_CPK_cor_clonality.png,
                 corrected_CDR3_nt_length_distribution.png,
corrected_CDR3_aa_length_distribution.png,
                 corrected_top10_IGHV_genes.png, corrected_top10_IGHJ_genes.png,
                 log_corrected.txt, scripts/corrected_analysis.R
```

8.3 Summary

This module is a one-minute R analysis over 11 input files plus a 375-sample cohort, and it is the first module in this course where the tutorial's own definitions, rather than its commands, are the weak point. Nine issues were verified and recorded. The three that change the numbers are: `nrow()` used as a richness measure when TRUST's report is not keyed by CDR3 (TBCR-02), a blacklist that removes `partial` but not `out_of_frame` (TBCR-03), and a cohort loop that skips the quality filter Part 3 applies (TBCR-08). The corrected pass halves the median diversity (918 to 446) while preserving sample ranking (Spearman 0.9975), so relative comparisons survive and absolute cohort-level statements do not. One sample yields `NaN` clonality in the as-written run and three have no valid IGH clone at all after correction; all are reported as undefined rather than imputed.

Appendix A — Deviation log

#	Tutorial	This run	Reason
1	interactive `R`, state carried between Parts	one `Rscript` per Part, state re-derived	reproducible, logged
2	outputs into `~/airr_output`	outputs into `results/NN_*/` via `setwd()`	repository layout
3	Part 0 SSH + `.bashrc` + `conda init`	`conda activate airr` only	already on the cluster
4	absolute input paths	unchanged	read-only course share

#	Tutorial	This run	Reason
5	no diagnostics	extra `[diag]` prints	evidence for this document
6	`p` without `print()` in three plot blocks	unchanged in as-written, `print()` in corrected	fidelity in as-written run
7	tutorial ends at three correlation plots	plus corrected pass, TRB extension, comparison	explicitly requested
8	filename `diveristy_cor_entropy.png`	kept as written; corrected copy spelled correctly	fidelity

Appendix B — Reproduction

```
source /lustre1/share/miniconda3/etc/profile.d/conda.sh
conda activate airr
export LC_ALL=C LANG=C

cd /gpfs1/home/bjx131_pkuhpc/class/TBCR-analysis/results
Rscript 01_clonotype/scripts/part1_clonotype.R    2>&1 | tee 01_clonotype/log_part1.txt
Rscript 02_gene_usage/scripts/part2_gene_usage.R  2>&1 | tee 02_gene_usage/log_part2.txt
Rscript 03_cdr3_length/scripts/part3_cdr3_length.R 2>&1 | tee 03_cdr3_length/log_part3.txt
Rscript 04_diversity/scripts/part4_diversity.R    2>&1 | tee 04_diversity/log_part4.txt
Rscript 05_corrected/scripts/corrected_analysis.R 2>&1 | tee 05_corrected/log_corrected.txt
```

Total wall time: about 45 s (the cohort loop is 14 s, the corrected pass 24 s). No scheduler is used; the analysis is single-threaded and fits comfortably on the login node.

Expected key numbers, for verification: 974 rows in `cdr3.out`; 745 BCR assemblies; 930 report rows; IGHV top gene `IGHV3-15*01` at 0.1560; 169 partial rows out of 354; diversity 354 for the demo sample; 375 cohort rows; median raw diversity 918; median corrected diversity 446; one `NaN` clonality as written and nine undefined after correction.